

The logo of the University of Jordan is a shield-shaped emblem. It features a blue open book at the base, with a blue torch rising from the center. The shield is outlined in orange and yellow. The text "Software Testing – 4 – WBT Techniques" is overlaid on the logo.

Software Testing – 4 – WBT Techniques

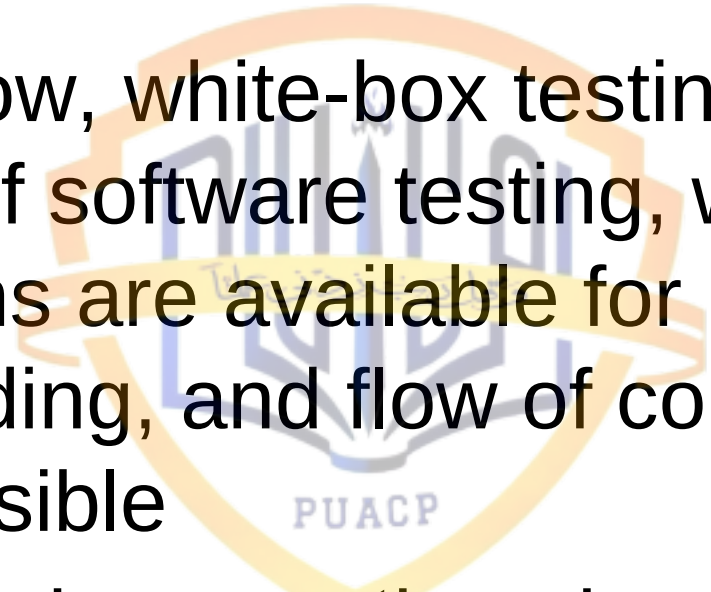
Lecture # 26

Recap

- Discussed black-box testing and white-box testing, and spent some time talking about their differences with respect to perspective, objects, timeline, defect focus, defect detection and fixing, techniques, and testers

Today's Lecture

- We'll have some more discussion on white-box testing
- Specifically, we'll talk about different white-box testing techniques

- 
- 
- As you know, white-box testing deals with that form of software testing, where code or programs are available for review, understanding, and flow of control and data are visible
 - Let's first ask a question about white-box testing

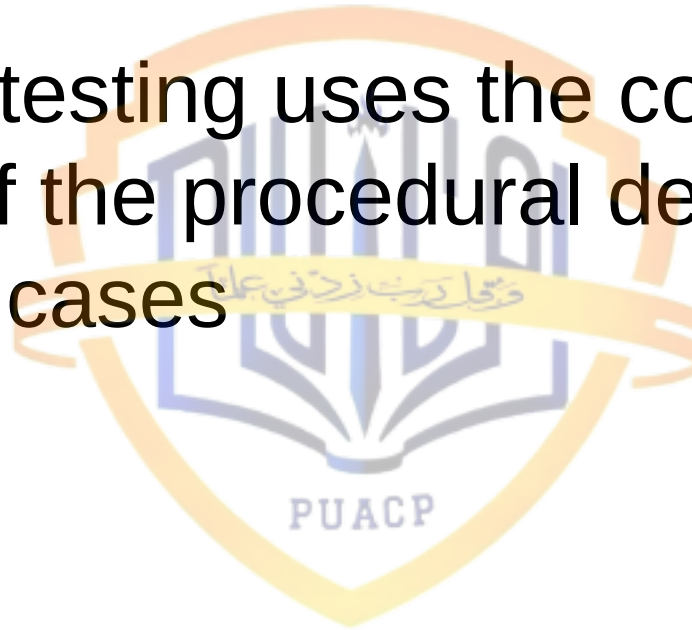
White-box Testing Question

- Why spend time and energy worrying about (and testing) logical details when we might better expend effort ensuring requirements have been met? **or**
- Why don't we spend all of our energy on black-box tests?

Answers to that Question

- Logical errors and incorrect assumptions are inversely proportional to probability that a program path will be executed
- We often believe that a logical path is not likely to be executed when, in fact, it may be executed on a regular basis
- Typographical errors are random, it's likely that untested paths will contain some

- White-box testing uses the control structure of the procedural design to derive test cases



WBT Methods Derive Test Cases

- Guarantee that all independent paths within a module have been exercised at least once
- Exercise all logical decisions on their true and false sides
- Execute all loops at their boundaries and within their operational bounds
- Exercise internal data structures to ensure their validity

White-Box Testing Techniques

- Basis path testing
- Condition testing
- Data flow testing
- Loop testing



Basis Path Testing - 1

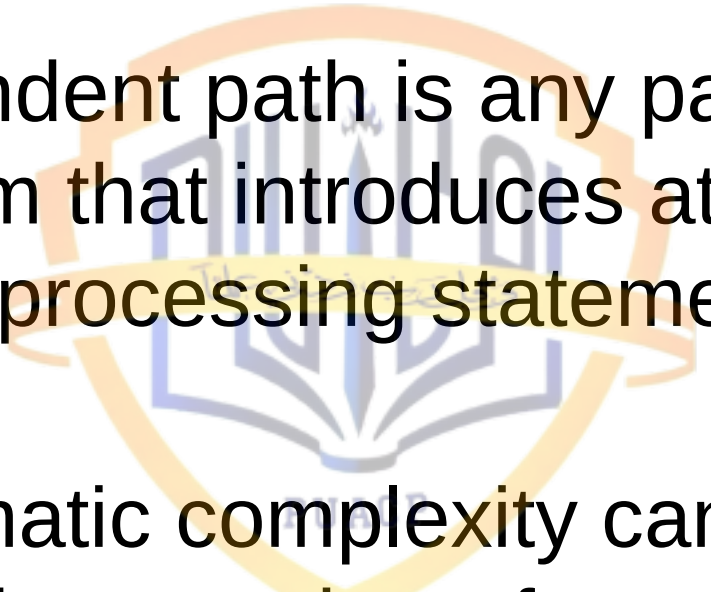
- Basis path testing is a white-box testing technique first proposed by Tom McCabe
- It enables the test case designer to derive a logical complexity measure of a procedural design and use this measure as a guide for defining a basis set of execution paths

Basis Path Testing - 2

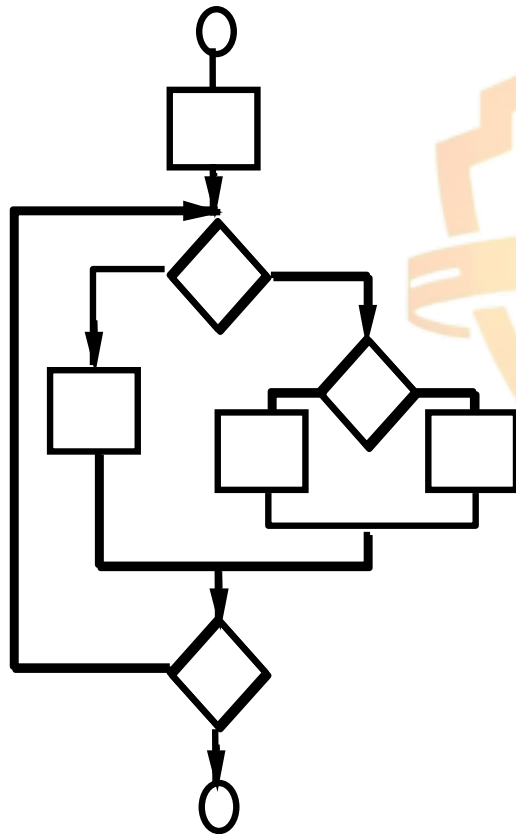
- Test cases derived to exercise basis set are guaranteed to execute every statement in the program at least once
- Basis path testing uses cyclomatic complexity, which is a software metric that provides a quantitative measure of the logical complexity of a program

Basis Path Testing - 3

- When the cyclomatic complexity used in the context of the basis path testing method, the value computed for cyclomatic complexity defines the number of independent paths in the basis set of a program and provides us with an upper bound for the number of tests that must be conducted to ensure that all statements have been executed at least once

- 
- 
- An independent path is any path through the program that introduces at least one new set of processing statement or a new condition
 - The cyclomatic complexity can be calculated in a number of ways

Cyclomatic Complexity

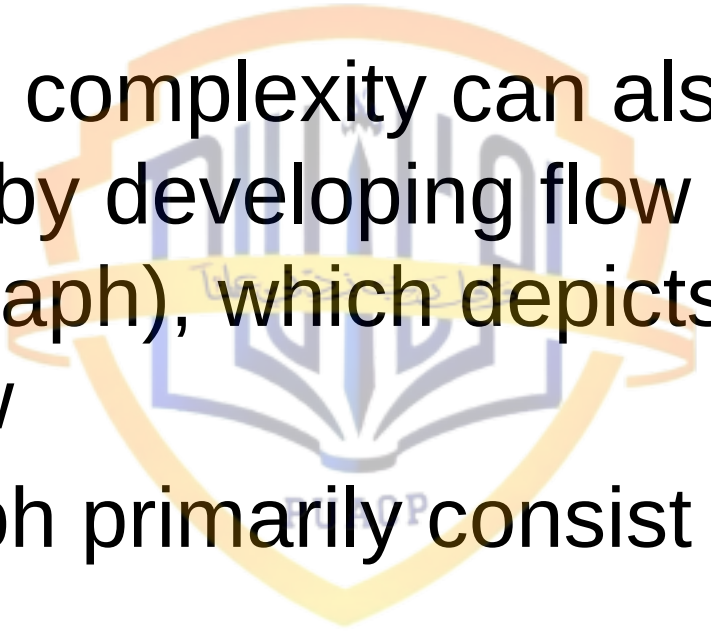


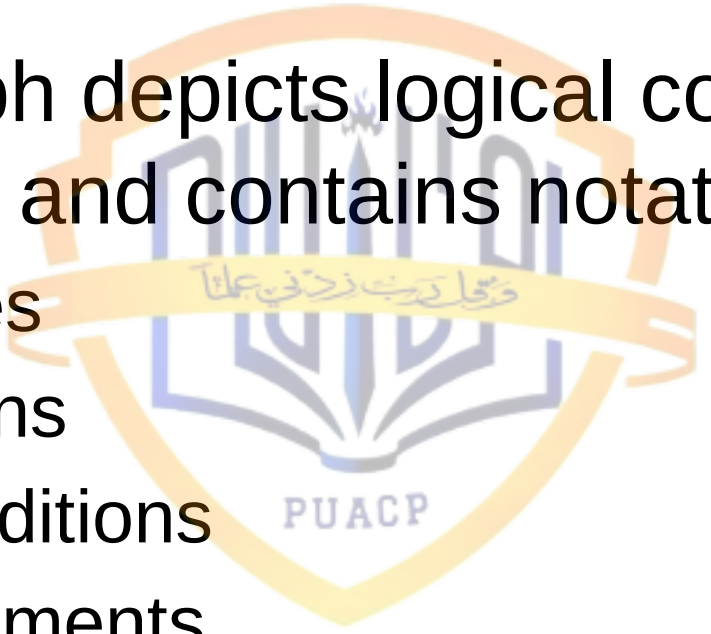
Number of simple decisions + 1

or

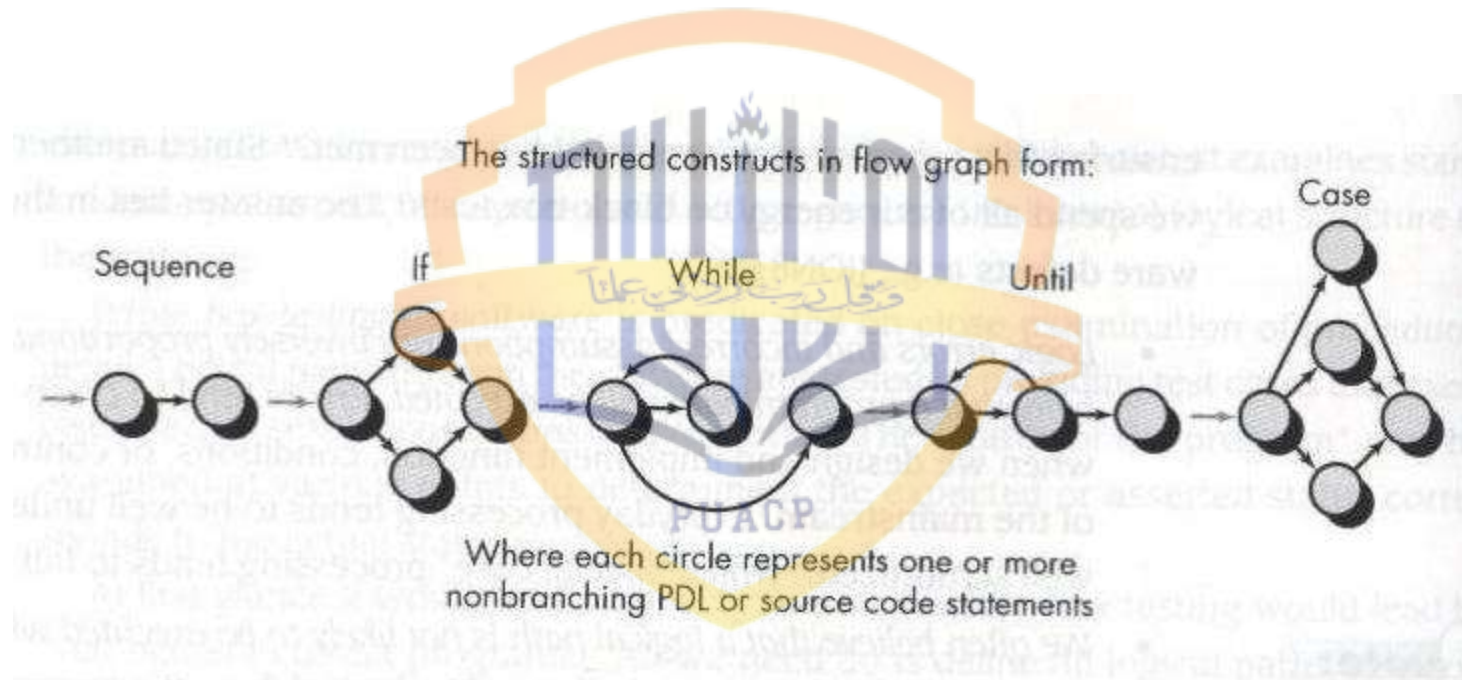
Number of enclosed areas + 1

In this case, $V(G) = 4$

- 
- 
- Cyclomatic complexity can also be calculated by developing flow graph (or program graph), which depicts the logical control flow
 - A flow graph primarily consist of edges and nodes

- 
- 
- A flow graph depicts logical control flow of a program, and contains notation for
 - Sequences
 - If conditions
 - While conditions
 - Until statements
 - Case statements

Flow Graph Notation

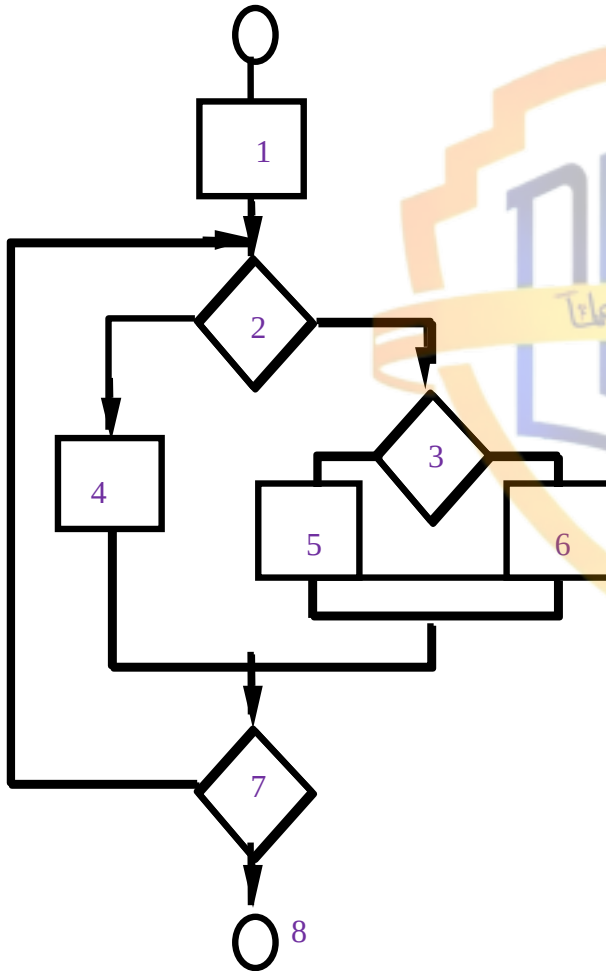


- If we are using flow graph graphs, the cyclomatic complexity is calculated as
 - $V(G) = \text{Edges} - \text{Nodes} + 2$

Steps in Basis Path Testing to Derive Test Cases

- Using the design or code as a foundation, draw a corresponding flow graph
- Determine the cyclomatic complexity of the resultant flow graph
- Determine a basis set of linearly independent paths
- Prepare test cases that will force execution of each path in the basis set

Finding Independent Paths



Since $V(G) = 4$,
there are four paths

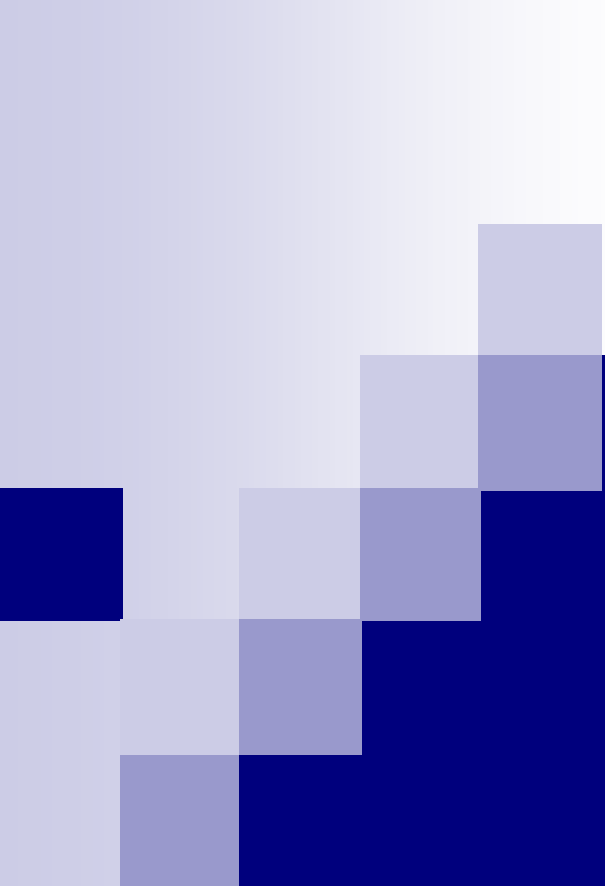
Path 1: 1,2,3,6,7,8

Path 2: 1,2,3,5,7,8

Path 3: 1,2,4,7,8

Path 4: 1,2,4,7,2,4,...7,8

Finally, we derive test cases to exercise these paths.



Conditional Testing



Condition Testing - 1

- Condition testing is a test case design method that exercises the logical conditions contained in a program module
- Conditions can be
 - Simple conditions
 - Relational expressions
 - Compound conditions

Condition Testing - 2

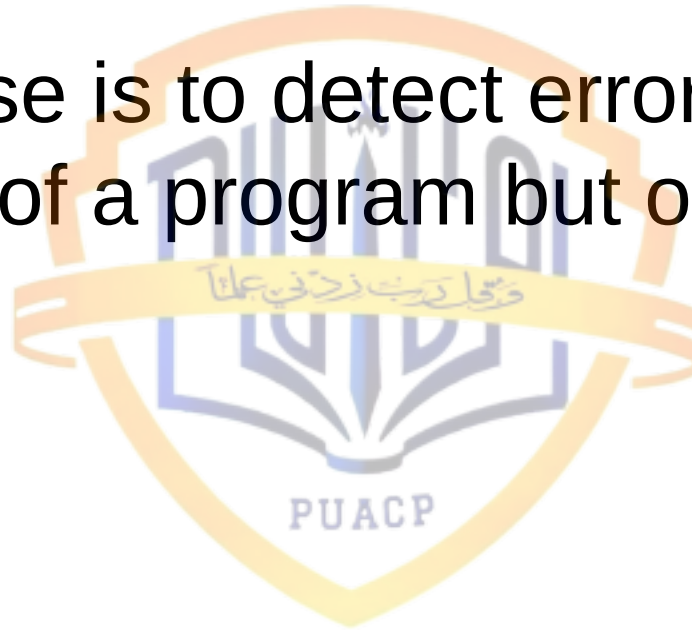
- Errors are much more common in the neighborhood of logical conditions than they are in the locus of sequential procession statements
- The condition testing method focuses on testing each condition in the program

Condition Testing - 3

- If a condition is incorrect, then at least one component of the condition is incorrect, therefore, types of errors in a condition include the following
 - Boolean operator error
 - Boolean variable error
 - Relational operator error
 - Arithmetic expression error

Condition Testing - 4

- The purpose is to detect errors in the conditions of a program but other errors also



Condition Testing - 5

- There are two advantages of condition testing
 - Measurement of test coverage of a condition is simple
 - Test coverage of conditions in a program provides guidance for the generation of additional tests for the program

Condition Testing Strategies

- Branch testing
 - For a compound condition, C, the true and false branches of C and every simple condition in C need to be executed at least once
- Domain testing
 - Testing related to relational operators
- BRO (branch and relational operator) testing

Data Flow Testing

- The data flow testing method selects test paths of a program according to the locations of definitions and uses of variables in the program
- Data flow testing strategies are useful for selecting test paths of a program containing nested if and loop statements

Loop Testing



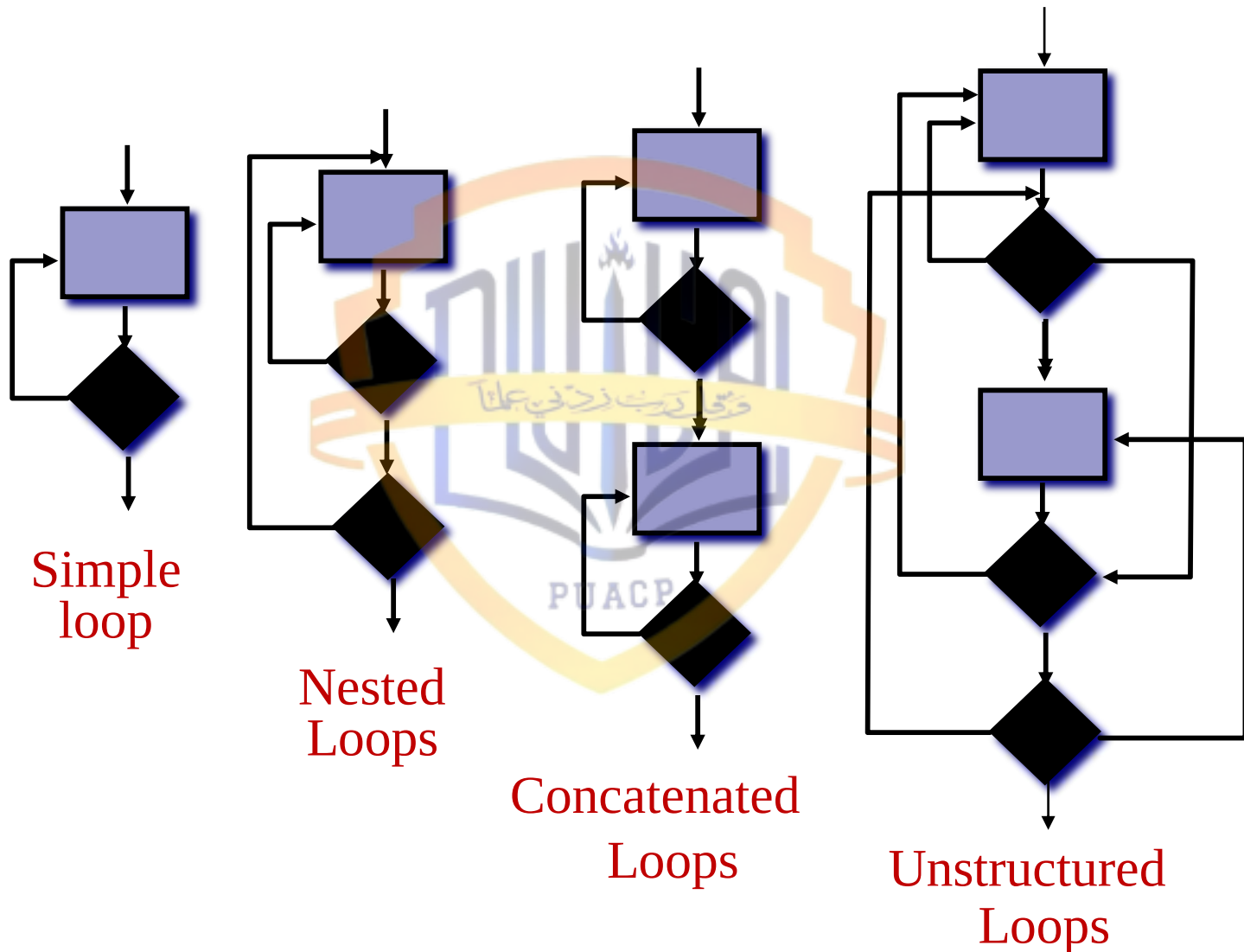
Loop Testing - 1

- Loops are the cornerstone for the vast majority of all algorithms implemented in software, and yet, we often them little heed while conducting software tests
- Complex loop structures are another hiding place for bugs. It's well worth spending time designing tests that fully exercise loop structures

Loop Testing - 2

- It is a white-box testing technique that focuses exclusively on the validity of loop constructs
- Four different classes of loops can be defined
 - Simple loop
 - Concatenated loops
 - Nested loops
 - Unstructured loops

Loop Testing



Loop Testing: Simple Loops

Minimum conditions—Simple Loops

1. skip the loop entirely
2. only one pass through the loop
3. two passes through the loop
4. m passes through the loop $m < n$
5. $(n-1)$, n , and $(n+1)$ passes through the loop

where n is the maximum number of allowable passes

Loop Testing: Nested Loops

Nested Loops

Start at the innermost loop. Set all outer loops to their minimum iteration parameter values.

Test the min+1, typical, max-1 and max for the innermost loop, while holding the outer loops at their minimum values.

Move out one loop and set it up as in step 2, holding all other loops at typical values. Continue this step until the outermost loop has been tested.

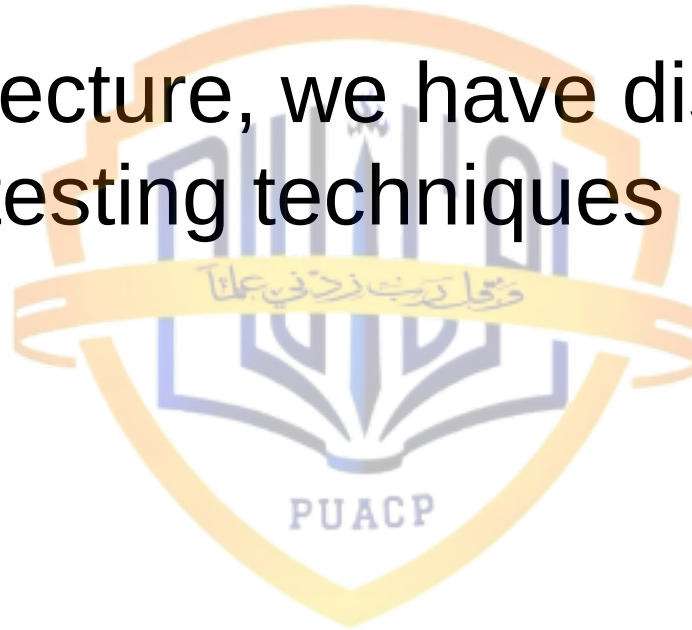
Concatenated Loops

If the loops are independent of one another
then treat each as a simple loop
else* treat as nested loops
endif*

for example, the final loop counter value of loop 1 is used to initialize loop 2.

Summary

- In today's lecture, we have discussed white-box testing techniques in quite a bit of detail



References

- Software Engineering: A Practitioner's Approach by Roger S. Pressman (Chapter 17.3-17.5)

